

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..20..2

CUSTOMIZATIONS

 Copy as Markdown

Customize anonymous sign-in

This page covers configuration options and UI customizations for the anonymous identity provider beyond the basic setup. On the server, you can tie sign-in to an app attestation token, react to new account creation, and adjust the rate limit. In the app, you can use the `AnonymousSignInWidget` to display the anonymous sign-in button in your own layout, or the `AnonymousAuthController` to build a completely custom authentication interface.

Server configuration

All server-side options for the anonymous provider are set on the `AnonymousIdpConfig`.

Using a token for app attestation

The anonymous `login` endpoint accepts an optional **token** that is forwarded to your `onBeforeAnonymousAccountCreated` callback. This lets you tie anonymous sign-in to an app attestation or app-check provider (e.g. [Firebase App Check](#)) so only requests from your real app can create anonymous accounts.

WARNING

Using the anonymous provider without a token for app attestation is not recommended due to the risk of abuse. Make sure to configure an attestation before releasing your app to the public.

Configuring the server

In `onBeforeAnonymousAccountCreated`, receive the optional **token** and verify it with your app-check provider. If verification fails or the token is missing (when you require it), throw an `AnonymousAccountBlockedException` with reason **denied** to block account creation.



```

AnonymousIdpConfig(
  onBeforeAnonymousAccountCreated: (
    Session session, {
      String? token,
      required Transaction? transaction,
    }) async {
    if (token == null || token.isEmpty) {
      throw AnonymousAccountBlockedException(
        reason: AnonymousAccountBlockedExceptionReason.denied,
      );
    }
    // Verify the token with your app-check provider (e.g. Firebase App Check).
    // Example: call Firebase's verifyAppCheckToken REST API or your provider's
    // verification endpoint. If invalid, throw AnonymousAccountBlockedException.
    final isValid = await _verifyAppCheckToken(session, token);
    if (!isValid) {
      throw AnonymousAccountBlockedException(
        reason: AnonymousAccountBlockedExceptionReason.denied,
      );
    }
  },
)

```

For Firebase App Check, you can verify the token from a custom backend using the [Firebase App Check REST API](#) (`verifyAppCheckToken`). Other app-check or attestation providers can be integrated the same way. The app sends a token, and the server validates it in the callback and denies creation if it is invalid.

For the app side of this flow, see [Configuring the Flutter app](#).

Reacting to anonymous account creation

Besides the `onBeforeAnonymousAccountCreated` callback to allow or deny creation, you can also use the `onAfterAnonymousAccountCreated` callback to run logic after a new anonymous account has been created (e.g. analytics or side effects).

```

AnonymousIdpConfig(
  onAfterAnonymousAccountCreated: (
    Session session, {
      required UuidValue authUserId,
      required Transaction? transaction,
    }) async {
    // e.g. track creation for analytics or send to your logging service
  },
)

```

Rate limiting

The anonymous provider includes built-in rate limiting per IP address to prevent abuse. The default is 100 anonymous account creations per hour per IP. You can customize the rate limit in the `AnonymousIdpConfig` using the `perIpAddressRateLimit` parameter:

```
// RateLimit is exported by the email provider library.
import 'package:serverpod_auth_idp_server/providers/email.dart';

AnonymousIdpConfig(
  perIpAddressRateLimit: const RateLimit(
    maxAttempts: 50,
    timeframe: Duration(hours: 1),
  ),
)
```

When the limit is exceeded, the provider throws an `AnonymousAccountBlockedException` with reason `tooManyAttempts`.

App configuration

On the app side, the main configuration is supplying the attestation token described in [Using a token for app attestation](#).

Configuring the Flutter app

Obtain a token from your app-check provider and pass it to the login call by setting `createAnonymousToken` on `AnonymousSignInWidget` or `AnonymousAuthController`. That callback is invoked when the user taps "Continue without account". The returned token is sent to the server as the `token` argument of the anonymous login endpoint.

```
import 'package:firebase_app_check/firebase_app_check.dart';
import 'package:serverpod_auth_idp_flutter/serverpod_auth_idp_flutter.dart';

AnonymousSignInWidget(
  client: client,
  createAnonymousToken: () async {
    // Get a Firebase App Check token (or similar) to prove the request comes
    // from your app to prevent abuse.
    final appCheckToken = await FirebaseAppCheck.instance.getToken();
    return appCheckToken;
  },
  onAuthenticated: () { /* ... */ },
  onError: (error) { /* ... */ },
)
```

Customize the sign-in button

When the button renders inside a `SignInWidget`, fields set on `buttonStyle` override its same-named arguments, as described in [Styling the buttons](#).

The `SignInWidget` uses the `AnonymousSignInWidget` internally when the anonymous provider is enabled. You can supply a custom `AnonymousSignInWidget` to the `SignInWidget` to override the default (e.g. to pass `createAnonymousToken` or change size and shape).

```
SignInWidget(  
  client: client,  
  anonymousSignInWidget: AnonymousSignInWidget(  
    client: client,  
    createAnonymousToken: () async ⇒ await getAppCheckToken(),  
    size: SignInButtonSize.medium,  
    shape: SignInButtonShape.rectangular,  
    // A custom widget replaces the built-in handling, so pass your own callbacks.  
    onAuthenticated: () { /* ... */ },  
    onError: (error) { /* ... */ },  
  ),  
)
```

Using the `AnonymousSignInWidget`

The `AnonymousSignInWidget` displays a single "Continue without account" button that starts the anonymous sign-in flow when pressed. You can customize the widget's behavior and appearance using its constructor parameters:

```
import 'package:serverpod_auth_idp_flutter/serverpod_auth_idp_flutter.dart';  
  
AnonymousSignInWidget(  
  client: client,  
  createAnonymousToken: () async {  
    // Optional: provide a token for app attestation (e.g. Firebase App Check)  
    return await getAppCheckToken();  
  },  
  onAuthenticated: () {  
    // Do something when the user is authenticated.  
    //  
    // NOTE: You should not navigate to the home screen here, otherwise  
    // the user will have to sign in again every time they open the app.  
  },  
  onError: (error) {  
    // Handle errors  
  },  
  // Button customization. The values shown are the defaults.  
  size: SignInButtonSize.large, // or medium, small  
  shape: SignInButtonShape.pill, // or rounded, rectangular  
)
```

Optionally, you can provide an externally managed `AnonymousAuthController` instance to the widget. A controller and a `client` are mutually exclusive, and `onAuthenticated` and `onError` belong on the controller in that case. Passing them alongside a controller trips an assertion, so a debug build throws.

```
AnonymousSignInWidget(  
  controller: controller,  
  size: SignInButtonSize.medium,  
  shape: SignInButtonShape.rectangular,  
)
```

Customizing the button appearance

The button renders flat, with no background fill and no border, and follows your app's theme brightness for its label color. It sets its own colors and corner radius, so a `TextButtonThemeData` does not reach it and an `ElevatedButtonThemeData` cannot change those. Properties the button leaves unset, such as `side` and `textStyle`, still fall through from that theme.

To change the label's text style, pass `textStyle` to the widget:

```
AnonymousSignInWidget(  
  client: client,  
  textStyle: const TextStyle(fontWeight: FontWeight.w600),  
)
```

Inside a `SignInWidget`, style every provider button at once with `buttonStyle` instead. See [Styling the buttons](#).

The button is at least 240 pixels wide and at most 400. Place it in a `SizeBox`, `Expanded`, or `Flex` to control layout.

Build a custom UI with AnonymousAuthController

For full control over the UI, use the `AnonymousAuthController` class. It provides the anonymous sign-in logic without any built-in widget, so you can trigger login from your own button or flow and build a completely custom layout.

```
import 'package:serverpod_auth_idp_flutter/serverpod_auth_idp_flutter.dart';  
  
final controller = AnonymousAuthController(  
  client: client,  
  createAnonymousToken: () async  $\Rightarrow$  await getAppCheckToken(),  
  onAuthenticated: () {  
    // Do something when the user is authenticated.  
  },  
  onError: (error) {  
    // Handle errors  
  },  
)
```

AnonymousAuthController state management

The controller notifies listeners when its state changes. Use these properties to drive your UI:

```
// Check if a request is in progress
final isLoading = controller.isLoading;

// Check current state (idle, loading, error, authenticated)
final state = controller.state;

// Listen to state changes
controller.addListener(() {
  setState(() {
    // Rebuild when controller state changes
  });
});
```

AnonymousAuthController methods

The controller exposes a single action for anonymous sign-in:

```
// Start anonymous sign-in.
// Obtains token if `createAnonymousToken` is set, then calls the login endpoint.
await controller.login();
```

Call `controller.login()` from your custom button's `onPressed`, or from any other trigger (e.g. after a delay or when the user performs an action). The controller handles loading state, success, and errors and invokes `onAuthenticated` or `onError` as appropriate.

TIP

Remember to dispose the controller when it is no longer needed (e.g. in your widget's `dispose`), unless the widget manages its own controller and disposes it for you.

Related

- [Setup](#): enable the anonymous provider on the server and show the sign-in button in your app.
- [UI components](#): the shared sign-in screen and styling for all provider buttons.
- [Working with users](#): access the authenticated user and profile data on the server.